# Computational Complexity

In the previous modules, we have discussed many different algorithms of various types for solving different problems. **Algorithm Design** concerns itself with the following question: Given a problem $P$, find an algorithm which solves $P$ with the best *cost*. For our discussions, the cost is usually associated with the time complexity with which the algorithm runs.

**Complexity Theory** is a theoretical domain which deals with classifying problems into different **classes**, based on their use of some computational resource (e.g. time, space, etc). It is a branch of a broader subject called **Theory of Computation** which deals with what problems can be solved on a model of computation using an algorithm, and how efficiently they can be solved.

## Introduction

We are interested in classifying problems into classes based on their time complexity. So we can define a **complexity class** as:

$\text{TIME}(t(n))$ := set of problems which have a worst case deterministic $t(n)$ time algorithm

But, what is a problem? What is an algorithm? What is $n$?

### Problems

A computational problem, $P$ is a binary relation from a set of input instances $I$, and a set of solutions $O$.

$$P : I \to O$$

For example, consider the SHORTEST-PATH problem: Given a graph $G$ and two vertices $s$ and $t$, find a shortest path from $s$ to $t$. An input instance $i \in I$ of this problem is the graph and two vertices, $i = \langle G, s, t \rangle$. An output instance $o \in O$ would be a sequence of vertices from $s$ to $t$ (if one exists, and perhaps an empty sequence when none do).

### Representation of the input space

For the purposes of running an algorithm on a "computer", we wish to represent the input instances using a **discrete** and **finite** encoding. We formulate this using formal languages:

Let $\Sigma$ be an alphabet (a finite set of symbols). Then, any input instance is represented as a string over $\Sigma$, $w \in \Sigma^*$. The input size $n$ is the length of this string, $n = |w|$. For example, in usual settings, $\Sigma = \{0, 1\}$ is chosen and the input instances are encoded as binary strings, which are given to the algorithm as input.

Though it might be obvious, **finiteness** is an important requirement for the representation of the input instance. Consider the problem of checking whether a given regular language is empty or not. Now, an input instance to this problem is any regular language, but regular languages are (usually) infinite objects. So we would need to represent the input instance as a finite object, e.g. an automaton.

Another assumption we make, is that when describing a problem, we may not explicitly define the exact scheme we use for encoding an input instance. We will assume some **reasonable encoding scheme** is used. It turns out that the time complexity of algorithms depends on the encoding used, but in practice, if we rule out 'expensive' encoding schemes (see box below), then the usual encoding schemes are all equivalent.

**Notion of a reasonable encoding**

In 2002, Prof. Manindra Agrawal and students Neeraj Kayal and Nitin Saxena from IIT Kanpur released a nine-page paper titled "PRIMES is in $P$"[1]. This paper showed a groundbreaking result, proving for the first time that checking if a given number is prime can be done deterministically in **polynomial time**. (We define the complexity class **P** shortly below.) This is widely regarded as the most impactful research coming out of India in the last few decades in the area of theoretical computer science. The paper was published in *Annals of Mathematics* in 2004 and was awarded both the Gödel Prize and the Fulkerson Prize, along with the Clay Research Award.

But, isn't the naive way of checking if a number is prime (trial division) itself a polynomial time algorithm? Given a number $n$, check its divisibility with all numbers $m$ from $2 \leq m \leq \sqrt{n}$. If it is divisible by any of them, it is not prime, otherwise it is. So this algorithm requires $O(\sqrt{n})$ steps to find the solution, which is indeed (better than, even) polynomial time. So why was the above discovery so groundbreaking? What are we missing?

Let's look at it carefully: To know the time complexity of the above (abstract) algorithm of trial division, we need to carefully define the problem PRIMES, specifically its input encoding. What is an input instance of this algorithm? Just a number, $N$ (let's not call it $n$ as that means something else!). Now how are we encoding this input as a finite string? A usual choice is to encode it as a **binary** number. So for this choice of the input encoding, what is the input size $n$? It is the length of the string that represents an input, so for input $N$, we require $\lceil \log_2 N \rceil$ bits. So, $n = \lceil \log_2 N \rceil \Rightarrow N = O(2^n)$. Our trial division algorithm takes $O\left(\sqrt{N}\right) = O\left(\sqrt{2^n}\right) = O(2^{n/2})$, which is actually **not polynomial** in $n$, the **input size**. For the time to be polynomial in $n$, it should actually be polynomial in $\log_2 N$ and not involve $N$ or higher order terms. The algorithm that AKS show has this property, and thus they claim that they show an algorithm to solve PRIMES in polynomial time.

Okay, but what about other input encodings? Observe that encoding in any base $b$, $b \geq 2$ are all equivalent for the purposes of the time complexity — the input size only differs by a constant multiple, so the time complexity with respect to the input size is same up to a constant multiplier. But, what if we encode the number $N$ in unary, i.e. the number $N$ is represented as a string of $N$ 1s? If we do so, then the input size $n = N$, and thus the time complexity of solving this problem is $O(\sqrt{n})$, so now we can claim that this algorithm solves this problem ($\text{PRIMES}_{\text{UNARY}}$) in polynomial time. But this claim is not useful: we simply made our input representation larger so that our time complexity with respect to the input size is smaller. We say, the unary encoding for numbers is not a **reasonable encoding**, as it is exponentially larger than a normal base $b$ encoding of the number. So, when we define problems such as PRIMES, we won't explicitly define what encoding we are using, and assume any reasonable encoding of the corresponding input instances. For other kinds of mathematical objects like graphs, etc., the commonly used representations (e.g adjacency list) are reasonable encodings.

The final thing that we haven't considered yet is what is an algorithm. Of course, an algorithm is just a procedure, a sequence of steps to perform, to get the solution from the given input. But for this to make sense, we need a formal **model of computation**. Historically, the Turing Machine was developed as such a formal model of computation, but it turns out that most reasonable models of computations are equivalent (the strong Church-Turing thesis), which allows us to talk about 'computability' in general.

Now we can return to the definition we saw:

$\text{TIME}(t(n))$ := set of problems which have a worst case deterministic $t(n)$ time algorithm

[1] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena, "PRIMES is in P," Annals of Mathematics 160, no. 2 (2004): 781–793, https://doi.org/10.4007/annals.2004.160.781. Read the paper here.

For a problem $P$, let $A$ be an algorithm solving $P$. Its worst case running time is said to be $O(f(n))$ if it takes at most $O(f(n))$ time on any input instance. Then, $P \in \text{TIME}(f(n))$. So we have these complexity classes: $\text{TIME}(n)$: all problems that can be solved in linear time, $\text{TIME}(n^2)$: all problems that can be solved in quadratic time (worst case), and so on. (Remember, $n$ is the size of the input representation.) We have a hierarchy of these complexity classes, like $\text{TIME}(n) \subseteq \text{TIME}(n^2)$ and so on.

### Decision Problems

We are specifically interested in a subclass of problems, where the output is a binary yes/no answer, i.e. $O = \{0, 1\}$. These problems are called **decision problems**. So $P : \Sigma^* \to \{0, 1\}$ is a decision problem.

Now from formal language theory, a decision problem is simply a **language** over $\Sigma$, $L \subseteq \Sigma^*$, $L = \{x \in \Sigma^* \mid P(x) = 1\}$, comprising those input instances, for which the answer is YES. And solving the decision problem is equivalent to **determining membership** of a string in the language $L$.

For example, consider the problem PRIMES which is the decision problem of determining whether a given input number is prime or not. Then, the corresponding language is

$$\text{PRIMES} = \{x \mid x \text{ is prime}\}$$

(where $x$ is encoded suitably as per our input encoding.) We use the same name PRIMES to both refer to the problem, and the corresponding language interchangeably for convenience.

We will primarily work with decision problems, since they have a clean analogy to formal languages. Many problems are not decision problems. But as we will see later, this is not a big restriction, most such general problems may be cast to a decision version of the problem, and by doing so we can analyse the hardness of these problems as well.

## The Class P

Historically, the theoretical ideas about computation (the Turing Machine model) led to **computability theory**. Computability deals with what problems can we solve (write an algorithm for that runs in finite time), and are there problems for which we can't write an algorithm? (There indeed are.) Now, in **complexity theory**, out of all the problems which we *can* **solve**, we want to talk about their **tractability**:

What problems can we solve "efficiently"?

Of course, efficiency is measured based on some computational resource we wish to optimise for, such as time, space, randomness, etc. We are dealing with time complexity here. But what is efficient, and what is inefficient?

We generally regard algorithms that run in **polynomial time** to be efficient. Thus, problems which have worst case polynomial-time algorithms are **tractable** (feasibly solvable on a computer). **Intractability** refers to the problems which cannot be solved in polynomial time. Usually, the known algorithms often require exponential time algorithms, which for larger input sizes, is infeasible to run on a computer.

The reason for this choice is philosophical, not mathematical (see box below). A supporting argument for choosing polynomial time problems to be tractable is the fact that for most reasonable models of computations, the notion of polynomial time solvability is equivalent with one another.

**What is a good notion for an "efficient" running time?**

Here is an excerpt from Edmonds' 1965 paper titled "Paths, trees, and flowers"[2] which describes his polynomial time algorithm for finding a maximum matching (the Edmonds' Blossom algorithm):

> "I am claiming as a mathematical result, the existence of a *good* algorithm for finding a maximum cardinality matching in a graph. There is an obvious *finite* algorithm, but that algorithm increases in difficulty *exponentially* with the size of the graph. It is by no means obvious whether or not there exists an algorithm whose difficulty only increases *algebraically* with the size of the graph. [...] One can find many classes of problems, besides maximum matching and its generalizations, which have algorithms of *exponential* order but seemingly none better. An example known to organic chemists is that of deciding whether two given graphs are isomorphic. For practical purposes the difference between *algebraic* and *exponential* order is often more crucial than the difference between *finite* and *non-finite*."
>
> — "Paths, Trees, and Flowers", Edmonds, 1965

Here, Edmonds asserts (one of the first to do so), that an algorithm is 'efficient', or 'good', if it runs in **polynomial time** in terms of the input size. Today, this notion is concretized; it is considered that the set of problems which have polynomial time algorithms, are the ones which are feasibly computable, or **tractable**.

### Definition

Define the class **P** as the class of all problems which have a worst case deterministic polynomial time algorithm. Formally,

$$\mathbf{P} = \bigcup_{k \in \mathbb{N}} \text{TIME}\left(n^k\right)$$

Some examples of problems in **P** are:

- PATH
- PRIMES
- Polynomial Factorisation
- Linear Programming
- Graph Planarity

## The Class NP

We have seen that problems in **P** can be *solved* efficiently. Now there are many problems, (focusing on decision problems only for now), which may or may not have an efficient solution, but can be *verified* efficiently, if given a correct *solution*.

Consider a decision problem, say HAMPATH, which is the problem of determining whether there exists a **Hamiltonian path** from $s$ to $t$ in a given graph $G$. A Hamiltonian path is just a path which goes through all vertices in $G$ exactly once. The corresponding language for this decision problem is defined as

$$\text{HAMPATH} = \{\langle G, s, t\rangle \mid \text{there exists a Hamiltonian path in } G \text{ from } s \text{ to } t\}$$

Now, it is not obvious if there exists any polynomial time solver for HAMPATH, i.e. whether HAMPATH $\in$ **P** or not. But there is a special property of this decision problem, it is **polynomial-**

[2] Edmonds, Jack. "Paths, Trees, and Flowers." Canadian Journal of Mathematics 17 (1965): 449–67. https://doi.org/10.4153/CJM-1965-045-4.

**time verifiable**. If we are given the input instance $\langle G, s, t \rangle$, and a solution path $p$, we can *verify* whether $p$ is a Hamiltonian path from $s$ to $t$ in $G$, in polynomial time. Though finding a Hamiltonian path is non-trivial, but *verifying* a solution can be done in polynomial time.

Formally, we can define a **verifier** $v_L$ for a language $L$ which takes two inputs $x$ and $y$, where $x$ is the input instance to the problem, and $y$ is another string called a **witness** (or a **certificate**). The verifier returns 1 or 0. The witness acts as proof of membership of the input instance $x$ in the language $L$. For some input $x$, if there exists some witness $y$ such that $v_L(x, y) = 1$, then $x \in L$. Otherwise, $x \notin L$.

In the above example, the verifier $v_{\text{HAMPATH}}$ will take two inputs, $x = \langle G, s, t \rangle$ (the input instance), and $y$ (the witness). It will check if $y$ is a valid path from $s$ to $t$ in $G$, and if it passes through every vertex in $G$ exactly once. If so, it returns 1, else it returns 0. Thus, this $v_{\text{HAMPATH}}$ is a verifier for HAMPATH as defined above:

- if for some input instance $x$ we can find a $y$ such that $v(x, y) = 1$, then that $y$ must be a Hamiltonian path in $G$, so $x \in$ HAMPATH
- if for some input instance $x$ there does not exist any $y$ such that $v(x, y) = 1$, then there must not be any Hamiltonian path in $G$, so $x \notin$ HAMPATH.

### Definition

Define the class **NP**[3] as the class of all **decision problems** which have a *efficient* verifier: For the verification to happen in polynomial time, we require that our witness $y$ itself must be at most polynomially larger than the input $x$.

A language $L$ belongs to **NP** if there exists a polynomial-time verifier $v_L$ and some constant $k$ such that

$$\begin{aligned} \forall x \in \Sigma^* \qquad & x \in L \implies \exists y, |y| \leq |x|^k, v_L(x, y) = 1 \\ & x \notin L \implies \forall y, |y| \leq |x|^k, v_L(x, y) = 0 \end{aligned}$$

In other words, $L \in$ **NP** if it has a verifier $v_L$ and:

- For every YES instance $x \in L$, there exists a short witness ($|y| \leq |x|^k$) such that $v_L(x, y) = 1$
- For every NO instance $x \notin L$, for every short witness $y$, $v_L(x, y) = 0$.

So, we can say, **NP** is the class of problems which have a **short effectively verifiable witness**.

Some examples of problems in **NP** are:

- HAMPATH; with the Hamiltonian path as a witness
- COMPOSITES; with a factorisation as a witness
- Graph Isomorphism; with a bijection over vertices as a witness

## Complement Classes

> **P = co-P**
>
> If $L \in \mathbf{P}$, then $\overline{L} \in \mathbf{P}$.

[3] Note that **NP** does not mean 'not polynomial' or something similar. It stands for 'nondeterministic polynomial time'; the class NP was originally studied in the context of nondeterminism, but we discuss an equivalent notion of verification.

**Proof**: Since membership in $L$ can be decided in polynomial time, non membership too can be decided in polynomial time (by simply flipping the outcome of the algorithm.) □

Define the class **co-NP** as:

$$\textbf{co-NP} = \left\{L \mid \overline{L} \in \textbf{NP}\right\}$$

In words, this is the class of languages whose *non-membership* can be efficiently verified using a short witness.

## Relation between classes

Now that we have defined these classes of problems, we can play with them. We can make some observations:

> **Theorem 1: P is in NP**
>
> $$\textbf{P} \subseteq \textbf{NP}$$

**Proof**: If $L \in \textbf{P}$, there is a polynomial-time algorithm to decide membership in $L$. Then, we can trivially create a verifier $v_L$ which simply invokes the solving algorithm itself. We can consider $\varepsilon$ (the empty string) as the correct witness for all $x \in L$, and for $x \notin L$ the verifier returns 0 for all witnesses. Thus, $L \in \textbf{NP}$. □

Similarly, we can show $\textbf{P} \subseteq \textbf{co-NP}$.

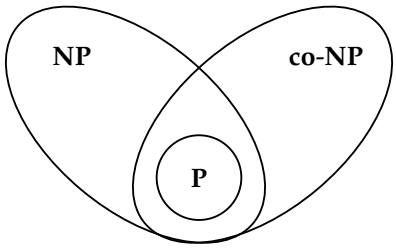



**Figure 1**: Currently believed relation between classes **P**, **NP** and **co-NP**

### P vs NP

Recall,

$$\textbf{P} = \text{problems which can be efficiently solved}$$
$$\textbf{NP} = \text{problems which can be efficiently verified.}$$

We know that HAMPATH $\in$ **NP**, i.e. it is efficiently verifiable. But, as of now, we do not know whether there exists any polynomial time solution for HAMPATH or not. So it seems, polynomial verifiability is a more general property than polynomial solving, or equivalently, **P** is a strict subset of **NP**. However, **we have not been able to prove (or disprove) this fact yet.**

The question of whether **P** = **NP** is a major unsolved problem in computer science. Solving this problem will have profound implications in many different domains, both theoretical and applied. It is one of the seven Millennium Prize Problems carrying a million dollar prize for solving it.

Most researchers believe, that the two classes are not equal, simply because people have invested enormous efforts in finding polynomial time algorithms for certain problems in **NP**, without success. Figure 1 shows the popularly believed relations between the classes **P**, **NP** and **co-NP**.

In this field, many fundamental questions beyond the $\mathbf{P} \neq \mathbf{NP}$ remain unsolved. For example, it is unknown whether $\mathbf{P} = \mathbf{NP} \cap \mathbf{co\text{-}NP}$ or not.

## Hardness of Problems

What entails proving $\mathbf{P} = \mathbf{NP}$? We will need to show some general method that every **NP** problem can be solved in polynomial time.

We make the following (abstract) claim:

> If we can solve the ***hardest*** problem in **NP** in polynomial time, then $\mathbf{P} = \mathbf{NP}$.

How can we formalise a notion of *hardness* between problems (as required in the above claim)? We want this notion to capture the idea, that if a harder problem can be solved, then an easier problem can be solved under the same constraints. This leads to the idea of **reductions** between problems.

### Reduction

A language $L_1$ is **polynomial-time mapping reducible**[4] to $L_2$, written as $L_1 \leq_p^m L_2$, if there exists a polynomial-time computable function $f : \Sigma^* \to \Sigma^*$, such that

$$\forall x \in \Sigma^* \qquad x \in L_1 \iff f(x) \in L_2$$

Note that the function $f$ has the property that it maps YES instances of $L_1$ into YES instances of $L_2$, and NO instances of $L_1$ to NO instances of $L_2$. If we can create such a function $f$, then we have a way to convert the input of one problem to another, thus being able to solve one of the problems by using the solver of another.

If $L_1 \leq_p^m L_2$, it means that the problem $L_1$ is **no harder than** the problem $L_2$. This is because, if we have a solver for $L_2$, we can create a solver for $L_1$ by first mapping the input of $L_1$ to an input of $L_2$ and using the solver of $L_2$ to get the answer. (Equivalently, we could say, problem $L_2$ is **at least as hard as** problem $L_1$.)

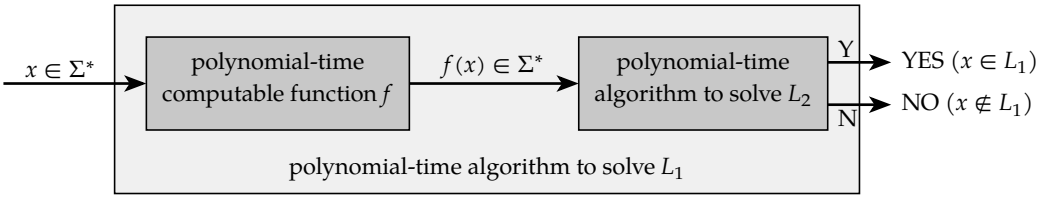



**Figure 2**: If $L_1 \leq_p^m L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$.

In particular, if $L_2$ is a polynomial-time solvable problem, then through this reduction we can also solve $L_1$ in polynomial time (thus we say $L_1$ is no harder than $L_2$).

> **Theorem 2: If $L_1 \leq_p^m L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$**

[4] Also known as a polynomial-time many-one reduction.

This is shown in Figure 2.

Another result is that this many-one reducibility relation is transitive. Problem 6 asks you to prove this.

> **Theorem 3: $\leq_p^m$ is transitive.**
>
> If $L_1 \leq_p^m L_2$ and $L_2 \leq_p^m L_3$, then $L_1 \leq_p^m L_3$.

So, to recap, if we have some *known easy problem* $L_2$ (which can be solved in polynomial time), and we are able to reduce some unknown problem $L_1$ into $L_2$, then this means that $L_1$ is no harder than $L_2$ and can also be solved in polynomial time.

## NP-Completeness

A language $L$ is NP-complete, if
1. $L \in \mathbf{NP}$,
2. $\forall L' \in \mathbf{NP}, \qquad L' \leq_p^m L$.

We denote the class of NP-complete problems as **NPC**.

In words, a NP problem $L$ is said to be NP-complete, if every NP problem reduces to $L$, i.e. every NP problem is no harder than $L$. Observe, that the class **NPC** exactly characterises our idea of the ***hardest*** problem in **NP**. Since every NP problem reduces to $L$, so $L$ is at least as hard as every other problem in **NP**.

We have an immediate observation:

> **Theorem 4: If any NP-complete problem is polynomial time solvable, then P = NP.**
>
> If $L \in \mathbf{NPC}$ and $L \in \mathbf{P}$, then $\mathbf{P} = \mathbf{NP}$.

**Proof**: Let $L \in \mathbf{NPC}$ and $L \in \mathbf{P}$. Then, for any NP problem $L' \in \mathbf{NP}$, $L' \leq_p^m L$ (by definition of NP-completeness). But, since $L \in \mathbf{P}$, this implies $L' \in \mathbf{P}$ (by Theorem 2). Thus we have shown $\mathbf{NP} \subseteq \mathbf{P} \Rightarrow \mathbf{P} = \mathbf{NP}$ (from Theorem 1). □

By contraposition, this also states, **if some NP problem is not polynomial-time solvable ($\mathbf{P} \neq \mathbf{NP}$), then no NP-complete problem is polynomial-time solvable**.

Since it is widely believed that $\mathbf{P} \neq \mathbf{NP}$, thus showing some problem to be NP-complete provides compelling evidence of its **intractability**.[5]

### NP-hardness

If a problem satisfies condition 2, regardless of it being in NP or not, then it is said to be **NP-hard**. All NP-complete problems are NP-hard. If a problem is NP-hard, it means it is *at least as hard to solve* as the problems in NP.

[5] Though we will often interchangeably use the terms NP-complete and *intractable*, these problems are not known to be intractable as of now. It is only if $\mathbf{P} \neq \mathbf{NP}$ (which is widely believed to be the case), then these problems are intractable.

## A first NP-complete problem

Till now we haven't shown any problem to be NP-complete. We need to show at least one NP problem to be in **NPC** (by showing that every NP problem reduces to it). The Cook Levin Theorem does this for the first time, establishing the existence of an NP-complete problem.

> **Theorem 5: Cook-Levin Theorem**
>
> CIRCUIT-SAT is NP-complete.

Proving this involves first showing that CIRCUIT-SAT is in **NP**, and then showing a reduction from every NP problem to CIRCUIT-SAT. We will define this problem and show the first part shortly. Showing the second part requires some technical detail about the model of computation used, so it is out of scope for this discussion.[6] We will assume the Cook Levin Theorem, and move forward.

## Proving NP-completeness for other problems

Now that we have one problem which we know to be NP-complete (CIRCUIT-SAT), we can use **polynomial-time mapping reductions**.

Recall, that $L_1 \leq_p^m L_2$ represents that $L_1$ is not harder than $L_2$, so if $L_2$ is easy, then $L_1$ must be easy as well. Now, instead of easiness, we want to prove *hardness*, so we use the other direction. If $L_1 \leq_p^m L_2$, and $L_1$ is *hard*, then $L_2$ must also be hard (since $L_2$ is at least as hard as $L_1$).

So, we have:

> **Theorem 6: Reducing a NP-complete problem to another problem**
>
> Let $L \in \mathbf{NPC}$. If for some $L'$, $L \leq_p^m L'$, and $L' \in \mathbf{NP}$, then
>
> $$L' \in \mathbf{NPC}$$

**Proof**: Since $L \in \mathbf{NPC}$, for all $L'' \in \mathbf{NP}$, we have $L'' \leq_p^m L$. We also have $L \leq_p^m L'$. By transitivity (Theorem 3), we have: for all $L'' \in \mathbf{NP}$, $L'' \leq_p^m L'$. Since $L'$ is NP as well, thus $L' \in \mathbf{NPC}$ (by definition). □

Thus, given a new problem $L$, the strategy for proving it to be NP-complete (*intractable*) is:

1. Show that $L$ is NP.
2. Choose a problem $X$ which is known to be NP-complete.
3. Prove that $X \leq_p^m L$. This involves describing a function $f$ which maps from every instance of $X$ to some instance of $L$, such that the YES instances of $X$ are mapped to YES instances of $L$, and vice versa. Also, $f$ should run in polynomial time.

This will be the content of the next module on **Intractability** where we will show various different problems to be NP-complete.

[6] Lemma 34.6 in CLRS book 3e, and Theorem 8.13 in KT book prove this in some level of detail.

## Satisfiability

Here, we define some foundational problems involving satisfiability of boolean formulas, which will be used for showing problems intractable. We start with some definitions:

A **boolean variable** is a variable which can take two values, 1 or 0.

A **boolean function** $f : \{0,1\}^n \to \{0,1\}$ is a mapping from $n$ boolean inputs to one boolean output.

A boolean function $f$ is said to be **satisfiable** if there exists some **assignment** $(a_1, a_2, ..., a_n) \in \{0,1\}^n$, such that $f(a_1, a_2, ..., a_n) = 1$.

We can represent a boolean function in many ways. One of the simplest ways we may represent it is a **truth table** (Figure 3 (a)). A truth table depicts the output of the function $f$ for all possible assignments of the variables. For a $k$-variable function, its truth table has $2^k$ rows.

If we wish to consider the problem of **Satisfiability**: given a boolean function, determine if there exists a satisfying assignment of variables; we need to choose what format the input is given in. Suppose we give the input as a truth table. Then, checking satisfiability is very easy: simply check if any output in the truth table is 1, if so, that is a satisfying assignment. But, the input size is the size of the truth table, which is $2^k$ (for a formula with $k$ variables). This is actually an *unreasonable representation scheme* of the boolean function: Most functions can be represented by using an exponentially smaller description, like a formula or a circuit.

| $x$ | $y$ | $f$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

$$f = (x \wedge \neg y) \vee (\neg x \wedge y)$$

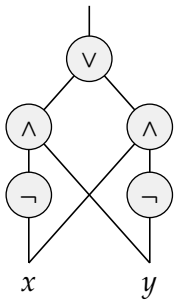



**Figure 3**: Representation of the function $x \oplus y$, or the parity of two variables, in different forms. (a) A truth table. (b) A boolean formula. (c) A boolean circuit.

A **boolean formula** (Figure 3 (b)) is an expression involving the input variables, and operators $\neg$, $\wedge$, and $\vee$ (and sometimes $\to$ and $\leftrightarrow$ as well), and parenthesis to set precedence. For a given assignment of variables, the output can be determined by substituting the value of each boolean variable and solving the resulting expression.

We can formulate a satisfiability problem: take as input a boolean formula $\phi$, and determine whether there exists a satisfying assignment for $\phi$. This problem is called SAT, or **Formula Satisfiability**. Observe that an instance of SAT (a boolean formula) can be represented as an expression tree, where the internal nodes are operators, and leaf nodes are the input variables.

Another way of representing a boolean function is by using a **boolean circuit** (Figure 3 (c)): a directed acyclic graph, with nodes representing boolean operations ($\wedge$, $\vee$, $\neg$) and input variables. Similar to an expression tree, the values of the variables are propagated through the circuit and the final value is determined.

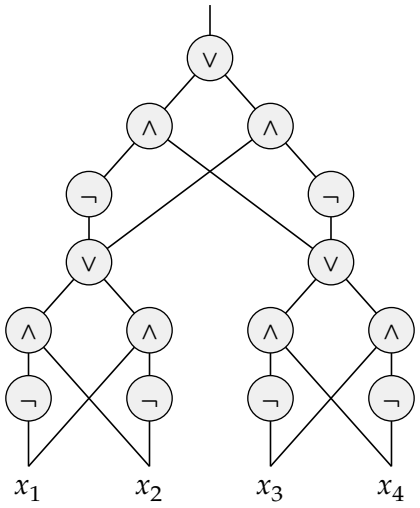



**Figure 4**: A boolean circuit for the four-input parity function, or $f = x_1 \oplus x_2 \oplus x_3 \oplus x_4$. The central $\vee$ nodes have a fan-out of 2.

Notice, that in a boolean circuit, the output of one operator can be reused by multiple higher level nodes. This is the difference between a boolean circuit and a boolean formula. For an internal operator node, define its **fan-out** as the number of outgoing edges it has. (Figure 4)

The corresponding satisfiability problem for boolean circuits is called CIRCUIT-SAT, i.e. given a boolean circuit, determine whether there exists a satisfying assignment of variables.

## SAT, CIRCUIT-SAT is in NP

For both of these problems, the short witness for verification is a **satisfying assignment of variables**. (Why is it short? If the function is of $k$ variables, the size of the formula or circuit will be $\Omega(k)$, so the witness (of size $\Theta(k)$) is bounded by a polynomial of the input size).

Thus the verifier $v_{\text{SAT}}$, given an instance $\phi$ and a witness $a$, verifies whether assigning the values $a$ to the variables and solving the expression results in 1. Similarly, the verifier $v_{\text{CIRCUIT-SAT}}$, given a circuit $C$ and a witness $a$ simulates the circuit over input $a$ and checks if it outputs 1. This can be done in polynomial time of the size of the input, so these problems are polynomial-time verifiable, and hence in **NP**.

Recall, from the Cook-Levin Theorem, we take that CIRCUIT-SAT is NP-Complete. We will use this in the next module to prove SAT, and many other problems NP-Complete as well.

## Self-Reducibility

Many problems of interest are often **optimisation problems**, or **search problems**, in which instead of a simple binary choice, we wish to find a (possibly best) feasible solution to the problem. Our discussion on hardness of problems (NP-completeness) was only confined to the realm of **decision problems**.

For example, as we will see, SAT is NP-Complete. What about SEARCH-SAT, where we have to find the satisfying assignment as well? We will show that these two problems are actually polynomially equivalent, and if one knows how to solve the decision version efficiently, they can also solve the search version efficiently. This is called **Self-Reducibility**.

### Self-Reducibility of SAT

Suppose we are given a polynomial-time oracle $\mathsf{SAT}(\phi)$ which takes a boolean function and answers yes or no depending on its satisfiability. Then, we can write a polynomial-time solver for $\mathsf{SEARCH\text{-}SAT}(\phi)$ where $\phi$ has variables $x_1, x_2, \ldots, x_n$ as follows:

1. Call $\mathsf{SAT}(\phi)$. If it answers NO, return NULL.
2. For $i = 1$ to $n$,
   - Call $\mathsf{SAT}(\phi|_{x_i=1})$ (substitute $\phi$ by setting $x_i = 1$).
   - If it answers YES, store $x_i = 1$ in the result, and update $\phi \leftarrow \phi|_{x_i=1}$.
   - If it answers NO, store $x_i = 0$ in the result and update $\phi \leftarrow \phi|_{x_i=0}$.
3. Return the stored assignment as a satisfying assignment.

Note that here, after each iteration we maintain that $\phi$ is satisfiable. This invariant leads to the correctness of the algorithm.

In the above procedure, we call the decision version solver $n + 1$ times, to solve an instance of the search version of the problem. Other than the calls to the oracle, the rest of the cost is polynomial time.

From this, we can conclude, that **SAT and SEARCH-SAT are polynomially equivalent**:[7] Searching for a satisfying assignment is no harder than deciding whether one exists, up to a polynomial factor. Searching is trivially at least as hard as deciding, so the two versions have essentially the same complexity. Thus, studying the decision version of the problem suffices, since both the problems are of equivalent hardness.

This turns out to be the case for many problems that we will study, as asked in Problem 4.

[7] Note that the reduction used here is not a **mapping reduction** as we have discussed earlier. This is a different kind of reduction where the outer procedure may call the oracle multiple times and also use its output to do some computation. This kind of reduction is referred to as a **Turing reduction**.

## Problems and Solution Sketches

3. [CLRS] Show that if an algorithm makes at most a constant number of calls to polynomial-time subroutines and performs an additional amount of work that also takes polynomial time, then it runs in polynomial time. Also show that a polynomial number of calls to polynomial-time subroutines may result in an exponential-time algorithm.

(a) Suppose the algorithm makes $c$ calls to polynomial time subroutines which take $O\left(n^{k_1}\right)$, $O\left(n^{k_2}\right), \ldots, O\left(n^{k_c}\right)$ time, and an additional $O(n^q)$ time. Inductively, each subroutine's input size is polynomial in $n$ (this is bounded by the original size of the input, plus the total time before calling of the subroutine), so therefore the time taken in each subroutine is also polynomial in $n$.

Therefore the total time taken is $O(n^q) + \sum_{i=0}^{c} O\left(p_i(n)^{k_i}\right)$ which results in a polynomial in $n$.

(b) Counter example: Consider a subroutine $S$ which takes a string $t$ and returns $t \cdot t$ (two copies of itself). The time complexity of $S$ is $O(n)$, i.e. linear to its input size.

Let the algorithm $A$ be

- $t \leftarrow$ "0"
- **for** $i = 1$ to $n$
  - $t \leftarrow S(t)$
- **return** t

Clearly, $A$ makes $n$ subroutine calls, each of which takes polynomial time (in its input). Finally it returns a string of size $2^n$, therefore the total time for $A$ is $2^n$, which is exponential in $n$.

4. Define decision and optimization versions of the following computational problems. Prove that if there exists a polynomial-time algorithm for the decision version of the problem, then there exists a polynomial-time algorithm for the search version of the problem.
   (a) Vertex Cover.
   (b) Hamiltonian Cycle.
   (c) Hamiltonian Path.
   (d) Longest Path.
   (e) Subset Sum.

**Vertex Cover**:

- Decision version: VERTEX-COVER – given a graph $G$ and an integer $k$, does there exist a vertex cover in $G$ of size at most $k$?
- Optimisation version: MIN-VERTEX-COVER – given a graph $G$, find a minimum vertex cover of $G$.

Suppose there exists a polynomial time oracle VERTEX-COVER$(G, k)$ which answers YES if there exists a vertex cover of size at most $k$ in $G$. Then, we can write a polynomial time algorithm MIN-VERTEX-COVER$(G)$:

1. Find the size of the minimum vertex cover $k$, by binary searching on VERTEX-COVER$(G, k)$ and finding the minimum $k$ for which the oracle answers YES.
2. Let $S \leftarrow \phi$.
3. While $G$ has an edge $\{u, v\}$, at least one of $u$ or $v$ must be in the vertex cover. Call the oracle as VERTEX-COVER$(G - u, k - 1)$, where $G - u$ denotes the induced subgraph of $G$ on vertex set $V \setminus \{u\}$.
   - If it returns YES, $u$ can be in a minimum vertex cover. Insert $u$ to $S$. Update $G \leftarrow G - u$.
   - If it returns NO, $v$ must be the part of the vertex cover. Insert $v$ to $S$. Update $G \leftarrow G - v$.
   - $k \leftarrow k - 1$.
4. Stop when $k = 0$ (all edges must have been covered). Return $S$.

(Show correctness and time complexity.)

Similarly, the other self reductions can be shown.

5. Prove that if **NP** $\neq$ **co-NP**, then **P** $\neq$ **NP**.

We need to show, **NP** $\neq$ **co-NP** $\Rightarrow$ **P** $\neq$ **NP**, or equivalently **P** $=$ **NP** $\Rightarrow$ **NP** $=$ **co-NP**.

Assume **P** $=$ **NP**. We know that **P** $\subseteq$ **co-NP**, so we have **NP** $\subseteq$ **co-NP**.

For any $L \in$ **co-NP**, $L' \in$ **NP** $=$ **P**. Therefore, $L \in$ **P**, so $L \in$ **NP**. Thus, we have **co-NP** $\subseteq$ **NP**.

Thus, we can conclude **NP** $=$ **co-NP**.

6. Show that many-to-one reducibility relation is transitive.

Let $L_1 \leq_p^m L_2$, this means there exists a (poly time computable) function $f : \Sigma^* \to \Sigma^*$, such that

$$\forall x \in \Sigma^* \quad x \in L_1 \iff f(x) \in L_2$$

Also, let $L_2 \leq_p^m L_3$, so there exists a (poly time computable) function $g : \Sigma^* \to \Sigma^*$, such that

$$\forall x \in \Sigma^* \quad x \in L_2 \iff g(x) \in L_3$$

Then, we can show that $L_1 \leq_p^m L_3$, by the poly time computable function $h(x) = g(f(x))$. We have

$$\begin{aligned} \forall x \in \Sigma^* \quad x \in L_1 &\iff f(x) \in L_2 \\ &\iff g(f(x)) \in L_3 \\ &\iff h(x) \in L_3 \end{aligned}$$

Thus, the many-one reducibility relation is transitive.